第 08 章

自适应学习与评估复盘

从一次作答追踪判分、掌握度和复习调度，再用可解释指标审视系统效果。

这一章完成后，你能够

- 读懂确定性判分与重复提交的保护逻辑
- 手算掌握度、复习间隔和自适应选题优先级
- 区分业务进度、检索指标与内容质量指标
- 运行离线实验并解释小样本结果的实际边界

学习前提

理解题目、知识点、来源与学习会话的关系；完成固定 Chain 与辅导 Agent 章节；了解 Python async/await、事务和基本比例计算

代码基准：8aa0ba5 + 本轮名称与分页修改。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

8.1 业务场景：为什么下一题会变，辅导聊天却不改分？ 3

8.2 判分规则：归一化表达，保留有意义的差异 5

8.3 一次提交：事务与幂等为什么在模型解释之前？ 6

8.4 掌握度：一个规则分数怎样逐步变化？ 8

8.5 复习调度：SM-2 风格规则怎样决定日期？ 10

8.6 自适应选题：先过滤，再打分，最后解释原因 11

8.7 评估名词：业务正确率与检索质量怎样区分？ 12

8.8 完整离线实验：亲自核对更新和指标 14

8.9 真实记录复盘：高分结果应怎样阅读？ 16

8.10 读懂进度页：真实名称、每页 5 条与累计统计 17

8.11 练习、参考思路与后续学习 18

8.12 综合练习与参考思路 19

8.13 源码阅读清单与下一步 20

8.1 业务场景：为什么下一题会变，辅导聊天却不改分？

题库解决“有什么内容可学”，学习服务解决“这一轮怎样练习”。学习者提交答案以后，程序判分、保存作答、更新知识点掌握度、安排复习，并在练习模式下重新选择下一题。这里的自适应是一套可解释的 Python 规则；辅导 Agent 查询已有事实并讲解，默认不修改成绩或掌握度。



图 8-1 在线学习闭环。辅导聊天不位于成绩写入路径。

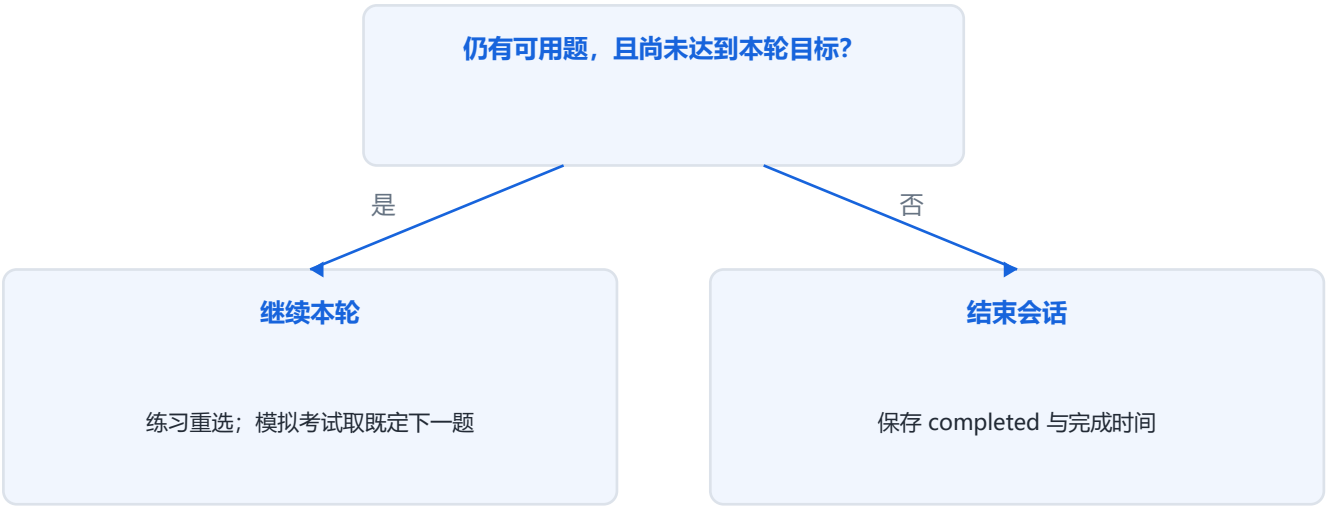


图 8-2 图 8-1a：继续或结束是条件分支，不是所有会话都必然产生下一题。



图 8-3 真实项目首页。观察资料、题库与学习入口怎样组成闭环；首页计数表示业务记录，不表示模型质量。

当前真实项目页面 / 只读现有业务数据 / 2026-10-12。界面数字只代表截图时的可见记录；没有开始学习或提交出题。

对象 / 名词	保存的事实	不能混为一谈的对象
StudySession	本轮模式、计划题数、已答数量和成绩	辅导聊天的 TutorConversation
AnswerRecord	一次作答的答案、成绩和提交标识	模型回复消息
MasteryRecord	知识点层面的规则分数与答题计数	人工考试能力或概率校准结果
ReviewTask	知识点复习到期时间和间隔参数	模型自行创建的学习建议
学习快照	可从数据库重建的当前页面状态	LangGraph 的运行检查点

表 8-1 本章使用的领域对象。

学习时先把这几个对象画在纸上：题目属于知识点，作答记录指向题目和知识点，掌握度与复习任务按知识点累计。这样就能理解同一知识点的不同题为什么共享学习状态，也能理解一次自由问答没有真实作答证据时，为什么不能直接宣布某个知识点已经掌握。

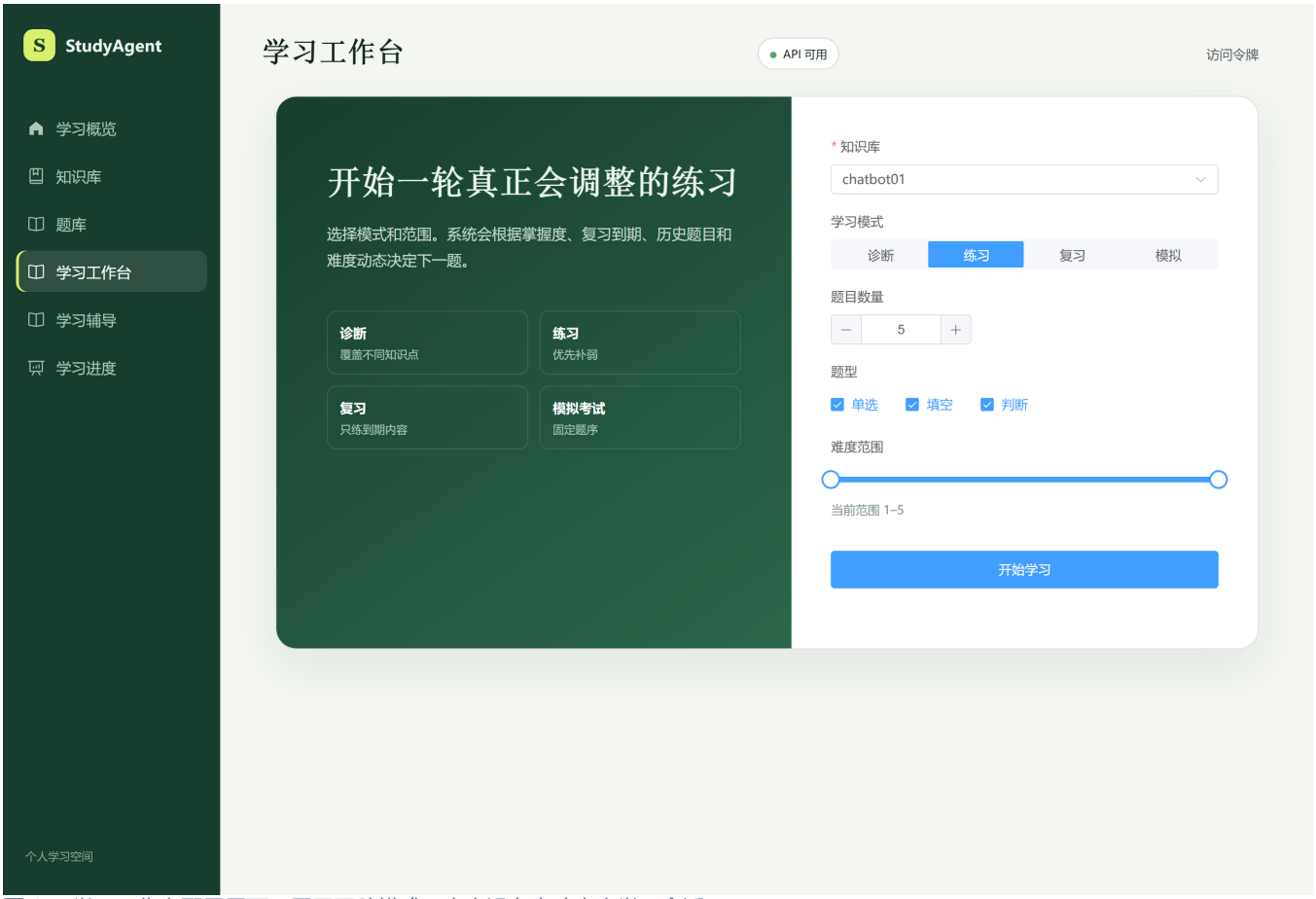


图 8-4 学习工作台配置界面，展示四种模式；本次没有启动真实学习会话。

当前真实项目页面 / 只读现有业务数据 / 2026-10-12。界面数字只代表截图时的可见记录；没有开始学习或提交出题。

8.2 判分规则：归一化表达，保留有意义的差异

grade_question 接收 Question 与用户答案，返回不可变 Grade：correct、score、max_score 和 feedback。当前只支持单选、填空与判断，正确得满分，错误为零，没有简答题语义判分或部分分。判断由程序完成；生成更丰富的错题解释不会改变已经确定的成绩。

源码节选：三种题型的确定性判分

定位：apps/api/study_agent/application/study.py，第 66-95 行。

```
66  def grade_question(question: Question, answer: object) -> Grade:
67      expected = _answer_list(question.correct_answer)
68      if question.question_type == "single_choice":
69          actual = str(answer).strip().upper()
70          accepted = {str(item).strip().upper() for item in expected}
71      elif question.question_type == "fill_blank":
72          actual = _normalize_text(answer)
73          accepted = {_normalize_text(item) for item in expected}
74      elif question.question_type == "true_false":
75          aliases = {
76              "true": "正确",
77              "1": "正确",
78              "正确": "正确",
79              "对": "正确",
```

代码续

```
            "false": "错误",
            "0": "错误",
            "错误": "错误",
            "错": "错误",
        }
        actual = aliases.get(_normalize_text(answer), _normalize_text(answer))
        accepted = {aliases.get(_normalize_text(item), _normalize_text(item)) for item in expected}
    else:
        raise DomainError("UNSUPPORTED_QUESTION_TYPE", "当前学习闭环不支持该题型", status_code=422)
    correct = bool(actual) and actual in accepted
    return Grade(
        correct=correct,
        score=question.max_score if correct else 0.0,
        max_score=question.max_score,
        feedback="回答正确" if correct else "回答错误，请结合解析复习该知识点",
    )
```

Question.correct_answer 被统一成列表，表示可接受答案集合。

单选转大写；填空使用 _normalize_text；判断将常用别名映射为中文标准值。

返回 Grade，而不是模型打分文本；不支持题型抛出 DomainError。

输入类别	实际规则	对照例子
单选	去首尾空格并转大写	a 与 A 相同；AB 不自动拆成多选
填空	NFKC、casefold、空白规范化、移除末尾句读	Ｒ Ａ Ｇ。与 rag 等价
判断	true/1/对映射正确；false/0/错映射错误	用户输入 True 可匹配标准答案“正确”
有意义字符	保留符号、小数点、斜线与内部空格	1/2 不等于 12；C++ 不等于 C

表 8-2 答案归一化不等于删除所有标点和字符。

NFKC 是 Unicode 的兼容规范化，可把全角字符转成常见半角形式。casefold 比普通 lower 更适合跨语言大小写比较。若希望“三次”与“3次”都判对，应把两者放进标准答案别名；现有函数不会理解任意同义改写。别名集合的质量也属于题库审核的一部分。

课程能力与项目实现

课程中模型可以承担开放题评审，但本项目尚未实现简答题、评分点匹配、判分置信度或人工复核分数。不要把设计文档中的 GradeResult 草案当成当前 Grade。

8.3 一次提交：事务与幂等为什么在模型解释之前？

网络重试可能让同一次答案到达两次。submit_answer 先对 submission_id 加锁并查历史记录：相同请求返回历史结果；同一个 ID 用于不同会话、题目或答案则报 SUBMISSION_ID_CONFLICT。它还锁定学习会话与相关资源，检查本次提交确实对应当前题，并且题目仍启用、有来源。



图 8-5 重复提交分支不再更新成绩、掌握度和计数；首次提交先完成业务事务。

源码节选：提交成绩后的讲解与回退

定位：apps/api/study_agent/application/study.py，第 324-342 行。

```
324         await uow.commit()
325         try:
326             explanation_data = (
327                 await explanation_generator.explain(
328                     question=question, user_answer=answer, correct=grade.correct
329                 )
330                 if explanation_generator
331                 else fallback_explanation(question=question, correct=grade.correct)
332             )
333         except Exception:
334             explanation_data = fallback_explanation(question=question, correct=grade.correct)
335         answer_record = await uow.answer_records.update_explanation(
336             answer_record.id, explanation_data
337         )
338         await uow.commit()
339         if not completed and study_session.mode == "mock_exam":
340             next_question, next_assignment = await self._current_assignment(uow, study_session)
341             next_reason = next_assignment.selection_reason if next_assignment else None
342         await self._sync_runtime_state(uow, study_session, next_question, state_store)
```

318 行之前已经创建作答、更新掌握度、复习任务和会话计数。

解释生成器为空或抛异常时，使用题库解析与绑定来源作为回退。

讲解结果单独保存，最后同步学习快照；模拟考试下一题来自既定顺序。



图 8-6 项目作答反馈页面。对照程序成绩、讲解和来源证据；截图若采用固定 HTTP 演示数据，按图中标注理解其数据来源。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09。真实答后反馈界面，固定 TCP 教学数据模拟一次错误选择，展示确定性判分、理解差距、原文证据与继续请教入口；没有真实模型调用。

解释生成器读取题干、用户答案、标准答案、已有解析和出题参考原文，按 source_kind 区分旧引文与新上下文，输出结论与差距。正确答案直接使用题库解析；错误答案在模型未配置或调用失败时也保留可读反馈。模型可以帮助表达，但只有一次选择题答错，不能可靠确定是概念混淆、粗心还是猜测，应把具体错因作为待核实假设。

答后来源与辅导引用的区别

答后讲解展示题目绑定的出题参考原文，新题为 reference_context；辅导 Agent 另走实时检索与证据复核。方案 B 改的是出题来源工作，不会移除 Agent 对证据 ID 和当前资料的校验。

8.4 掌握度：一个规则分数怎样逐步变化？

掌握度默认从 0.30 开始。正确时 M_new = M_old + 0.15 × (1 - M_old)，错误时 M_new = 0.8 × M_old，最后钳制到 0-1 并保留四位小数。这个更新逐渐趋近上限，不能把三次连续答对解释为已经百分之百掌握。它不使用 LLM，也没有按题目难度自动调整更新权重。

源码节选：掌握度更新与累计统计

定位：apps/api/study_agent/application/study.py，第 415-434 行。

```python
    async def _update_mastery(
        self, uow: UnitOfWork, question: Question, correct: bool, now: datetime
    ) -> MasteryRecord:
        current = await uow.mastery.get(question.knowledge_point_id)
        previous_score = current.mastery_score if current else 0.3
        answered = (current.answered_count if current else 0) + 1
        correct_count = (current.correct_count if current else 0) + int(correct)
        score = previous_score + 0.15 * (1 - previous_score) if correct else previous_score * 0.8
        return await uow.mastery.upsert(
            MasteryRecord(
                knowledge_point_id=question.knowledge_point_id,
                knowledge_base_id=question.knowledge_base_id,
                mastery_score=round(max(0.0, min(1.0, score)), 4),
                answered_count=answered,
                correct_count=correct_count,
                recent_accuracy=round(correct_count / answered, 4),
                confidence=round(min(1.0, answered / 10), 4),
                updated_at=now,
            )
        )
```

current 可能为空，首次使用 0.3；correct 是程序判分布尔值。

answered_count 与 correct_count 按知识点累计，更新后的对象由仓储 upsert。

recent_accuracy 当前计算 correct_count / answered_count，confidence 为 min(1, answered / 10)。

顺序 / 结果	掌握度计算	累计字段
初始	M = 0.3000	尚无作答记录
第一次正确	0.30 + 0.15 × 0.70 = 0.4050	正确率 1.0；confidence 0.1
第二次正确	0.405 + 0.15 × 0.595 = 0.4943	正确率 1.0；confidence 0.2
第三次错误	0.4943 × 0.8 = 0.3954	累计正确率 0.6667；confidence 0.3

表 8-3 按源码四位小数存储逐次计算，不能忽略中间舍入。

recent_accuracy 这个名称容易误导：当前实现实际是累计正确率，没有最近五次滑动窗口。confidence 只表示记录数量达到十次后的饱和值，不是统计置信区间，也不代表模型对答案的自信。阅读字段时应把计算方式与名称一起检查，避免在图表里赋予源码没有的含义。

旧方案公式如何处理

旧设计文档提出 0.70/0.20/0.10 混合公式和最近五次加权准确率，但它是目标设计。当前学习材料统一以 study.py 的实际更新为准，后续若改变算法，需要单独版本与回归验证。

删除知识库时使用软删除，数据库里可能仍保留历史会话、掌握度和复习任务。当前仓储查询会按 KnowledgeBase.status != deleted 过滤这些列表，因此概览和进度不应继续显示已删除知识库的旧记录。数据库保留与页面展示不是同一个操作。

仅删除题目不等于清空学习历史

删除题目会移除题目可用性，但所属知识库还有效时，历史学习记录仍有保留意义。想解释“为什么还有学习记录”，先分别核对知识库状态、题目状态与仓储过滤，不直接把所有历史统计删除。

8.5 复习调度：SM-2 风格规则怎样决定日期?

复习任务与知识点绑定。当前只把正确映射为 quality=5，错误映射为 quality=2，初始 ease_factor 为 2.5，下限为 1.3。ease 更新公式为 E_new = max(1.3, E_old + 0.1 - (5-q) × [0.08 + (5-q) × 0.02])。q 是本次质量等级，E 是用于扩展间隔的易记系数。

源码节选：复习质量、次数和间隔

定位：apps/api/study_agent/application/study.py，第 437-468 行。

```
437     async def _update_review(
438         self, uow: UnitOfWork, question: Question, correct: bool, now: datetime
439     ) -> ReviewTask:
440         current = await uow.review_tasks.get_by_knowledge_point(question.knowledge_point_id)
441         quality = 5 if correct else 2
442         ease = current.ease_factor if current else 2.5
443         ease = max(1.3, ease + (0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02)))
444         if correct:
445             repetitions = (current.repetitions if current else 0) + 1
446             if repetitions == 1:
447                 interval = 1
448             elif repetitions == 2:
449                 interval = 6
450             else:
451                 interval = max(1, round((current.interval_days if current else 1) * ease))
452         else:
453             repetitions = 0
454             interval = 1
455         return await uow.review_tasks.upsert(
456             ReviewTask(
457                 id=current.id if current else uuid4(),
458                 knowledge_point_id=question.knowledge_point_id,
459                 knowledge_base_id=question.knowledge_base_id,
460                 due_at=now + timedelta(days=interval),
461                 interval_days=interval,
462                 repetitions=repetitions,
463                 ease_factor=round(ease, 2),
464                 last_quality=quality,
465                 status="pending",
466                 updated_at=now,
467             )
468         )
```

先读取当前 ReviewTask，再更新 ease；后续间隔使用本次更新后的系数。

正确时连续次数加一，第一轮 1 天，第二轮 6 天，之后 round(旧间隔 × ease)。

错误时次数归零、间隔 1 天；due_at 从当前作答时间加间隔得到。

连续正确三次，系数依次是 2.6、2.7、2.8，间隔为 1、6、17 天；第三次取 round(6×2.8)。随后答错，系数减少 0.32，连续次数归零，次日复习。它没有提供六档用户自评质量，也没有根据作答耗时改变质量；elapsed_seconds 被保存，但没有进入此公式。



图 8-7 当前复习实现。每次作答更新同一知识点任务，而不是无限新增提醒。

复习不等于定时推送

review 模式查询 due_at 已到期的 pending 任务，再挑有来源的可用题。当前流程安排复习数据，并不等于已实现邮件、消息通知或自动唤醒。

8.6 自适应选题：先过滤，再打分，最后解释原因

AdaptiveQuestionSelector.select 接收知识库、模式、题型、难度范围、本轮已选题及上一次判分，返回 QuestionSelection 或 None。程序先排除已选题和无来源题；review 只保留到期知识点，diagnostic 优先选择本轮覆盖次数最少的知识点。没有候选时明确结束或报没有可用题，不让模型编造 ID。

模式 / 分量	当前生产规则	实际作用
practice / review 知识点分	0.45(1-M)+0.25D+0.10+0.20R	兼顾薄弱程度、到期和本轮刚答错
diagnostic 知识点分	0.20(1-M)+0.10D+0.70	配合最少覆盖次数过滤拓展知识点
题目分	0.45F+0.25N+0.20S+0.10T	难度匹配、新题、来源数和题型变化
总分	0.55×知识点分+0.45×题目分	分数降序，之后按稳定键打破平局

模式 / 分量	当前生产规则	实际作用
mock_exam	会话开始时固定题目与顺序	答错以后不临时换题或加题

表 8-4 权重以 adaptive_selection.py 为准，当前没有 goal_weight 学习目标项。

表中的 M 是掌握度，D 表示到期，R 表示上题在同一知识点答错。目标难度通常为 round(1+4M)，然后限制在请求范围；刚答错时降为上题难度减一。F = max(0, 1 - 难度差/4)，N 是未作答的新题标记，S = min(1, 来源数/2)，T 表示题型与上一题不同。来源数只是粗略代理，不是证据充分性的模型评分。

源码节选：题目特征、总分与追问惩罚

定位：apps/api/study_agent/application/adaptive_selection.py，第 96-116 行。

```
            target_difficulty = round(1 + mastery_score * 4)
            if recent_error and last_question is not None:
                target_difficulty = max(difficulty_min, last_question.difficulty - 1)
            target_difficulty = max(difficulty_min, min(difficulty_max, target_difficulty))
            difficulty_match = max(0.0, 1 - abs(question.difficulty - target_difficulty) / 4)
            freshness = 0.0 if question.id in answered_ids else 1.0
            source_quality = min(1.0, len(question.sources) / 2)
            type_diversity = float(
                last_question is None or question.question_type != last_question.question_type
            )
            question_priority = (
                0.45 * difficulty_match
                + 0.25 * freshness
                + 0.20 * source_quality
                + 0.10 * type_diversity
            )
            priority = 0.55 * knowledge_priority + 0.45 * question_priority

            same_point_count = selected_point_counts.get(question.knowledge_point_id, 0)
            if recent_error and same_point_count >= 3:
                priority -= 1.0
```

mastery_score、recent_error 在上面已经由数据库事实与上次判分计算。

Python round 参与目标难度与分数计算；所有因子来自现有题目和记录。

同点本轮已选至少三题且刚答错时扣 1.0 分，这是排序惩罚，不是禁止再选的硬上限。

手算示例：M=0.2、未到期、无刚答错，知识点分为 0.46。难度 2 的新题、一个来源、首次题型，题目分为 0.90，总分为 0.658。返回 reason 记录目标难度和各因子，便于解释为什么选它。没有掌握度记录时默认 0.3；reason.code 是解释优先级的标签，不是独立机器学习诊断。

8.7 评估名词：业务正确率与检索质量怎样区分？

业务进度告诉你已经练了多少题、得了多少分；检索评估问系统是否找到人工标注的相关片段；生成质量评估问回答是否由证据支持、是否切题。这三层需要不同输入。飞书课程采用“指标含义、公式、例子、结果解读”的节奏，本节沿用这个结构，并明确当前实际实现的是哪些指标。

指标	本项目计算方法	数值解释与边界
Hit@K	前 K 条至少命中一个相关片段的查询比例	一次命中不代表召回全部依据
Recall@5	每查询前五条命中数 / 标注相关数，再平均	按查询平均，空相关集合计 0
MRR	首个相关片段的排名倒数，再平均	当前函数在完整返回列表寻找首个相关项
P50 / P95	排序后取 ceil(n×比例) 的最近秩值	表示这组请求耗时，不是并发压测结果
Schema / 来源关联率	审计结构规则通过比例 / 有 sources 的比例	审计规则较宽；来源非空不保证引文支持
精确重复率	规范化题干重复的额外条数 / 总题数	忽略非词字符，不测语义重复

表 8-5 当前 evaluation/metrics.py 的指标口径。

源码节选：Hit、Recall 与 MRR 的逐样本计算

定位：apps/api/study_agent/evaluation/metrics.py，第 32-57 行。

```
    hits = {1: 0, 3: 0, 5: 0}
    recalls: list[float] = []
    reciprocal_ranks: list[float] = []
    latencies: list[float] = []
    for sample in samples:
        relevant = set(sample["relevant_chunk_ids"])
        retrieved = list(sample["retrieved_chunk_ids"])
        for k in hits:
            hits[k] += int(bool(relevant.intersection(retrieved[:k])))
        recall = len(relevant.intersection(retrieved[:5])) / len(relevant) if relevant else 0.0
        recalls.append(recall)
        first_rank = next((index for index, item in enumerate(retrieved, 1) if item in relevant), 0)
        reciprocal_ranks.append(1 / first_rank if first_rank else 0.0)
        latencies.append(float(sample["latency_ms"]))

    count = len(samples)
    return {
        "sample_count": count,
        "hit_at_1": round(hits[1] / count, 4),
        "hit_at_3": round(hits[3] / count, 4),
        "hit_at_5": round(hits[5] / count, 4),
        "recall_at_5": round(sum(recalls) / count, 4),
        "mrr": round(sum(reciprocal_ranks) / count, 4),
        "latency_p50_ms": percentile(latencies, 0.5),
        "latency_p95_ms": percentile(latencies, 0.95),
    }
```

每条样本需要相关片段 ID、实际召回列表与 latency_ms。

相关 ID 被转成集合，Recall 使用前五项交集；MRR 读取首个命中的顺序位置。

最后按样本数平均并舍入；空数据集的零值表示没有样本，不能当成系统实测性能。

RAGAS、Faithfulness 和 Answer Relevancy 可以帮助评价文本回答，但当前项目没有接入它们。来源验证与 evidence ledger 提供可追溯性，也不能直接等同于已测得的忠实度分数。若将来引入评判模型，应保存问题、召回内容、系统回答、标准答案及评判配置，再检查模型评分与人工判断的一致性。

审计器不等于生产门禁

question_is_valid 的单选审计允许至少两个选项；生产生成与启用要求 A-D 四项。因此 schema_valid_rate 的名称不能解释成完整生产 Schema 校验通过率。

8.8 完整离线实验：亲自核对更新和指标

第一个实验调用真实判分与两个更新函数，用内存仓储保存当前记录。MemoryRecord 只实现本实验访问的方法，SimpleNamespace 把两个仓储组合成最小依赖；它不模拟真实事务、并发锁或整个 submit_answer 会话，因此验证范围是纯规则结果，不是数据库可靠性验收。

完整可运行实验：判分、掌握度和复习更新

```python
import asyncio
from datetime import UTC, datetime
from types import SimpleNamespace
from uuid import UUID
from study_agent.application.study import StudyService, grade_question
from study_agent.domain.models import Question

class MemoryRecord:
    def __init__(self):
        self.record = None
    async def get(self, _):
        return self.record
    async def get_by_knowledge_point(self, _):
        return self.record
    async def upsert(self, record):
        self.record = record
        return record

question = Question(
    id=UUID(int=1), knowledge_base_id=UUID(int=2),
    knowledge_point_id=UUID(int=3), question_type="fill_blank",
    stem="检索增强生成的英文缩写是____。", options=None,
    correct_answer=["RAG"], scoring_points=[], explanation="缩写为 RAG。",
    difficulty=2, max_score=10.0, status="active",
    content_hash="offline-demo", vector_id=None,
)

async def main():
    uow = SimpleNamespace(mastery=MemoryRecord(), review_tasks=MemoryRecord())
    service = StudyService()
    now = datetime(2026, 10, 7, tzinfo=UTC)
    for index, answer in enumerate(["ＲＡＧ。", "rag", "未知"], 1):
        grade = grade_question(question, answer)
        mastery = await service._update_mastery(uow, question, grade.correct, now)
```

代码续

```
        review = await service._update_review(uow, question, grade.correct, now)
        print(index, grade.correct, mastery.mastery_score,
              review.interval_days, review.repetitions, review.ease_factor)
    print("accuracy =", mastery.recent_accuracy)
    print("confidence =", mastery.confidence)
    assert mastery.mastery_score == 0.3954
    assert review.interval_days == 1 and review.repetitions == 0

asyncio.run(main())
```

采用真实 Question 对象；全角输入经归一化判对。

依次正确、正确、错误，观察中间舍入和错误重置。

本章用私有方法隔离规则，仅用于教学；正式业务入口仍是 submit_answer。

结果解释：三行结果依次包含 True 0.405 1 1 2.6、True 0.4943 6 2 2.7、False 0.3954 1 0 2.38；累计正确率 0.6667，confidence 0.3。

第二个实验人工构造三条召回列表，手算一次后再执行。第一条首位命中且召回两个依据，第二条相关项排第二，第三条未命中；所以 MRR=(1+1/2+0)/3=0.5。耗时 10、20、100 是固定教学数字，不是服务器测量结果。

完整可运行实验：检索指标与宽松审计边界

```
from study_agent.evaluation.metrics import retrieval_metrics, question_metrics

samples = [
    {"relevant_chunk_ids": ["a", "b"],
     "retrieved_chunk_ids": ["a", "x", "b"], "latency_ms": 10},
    {"relevant_chunk_ids": ["c"],
     "retrieved_chunk_ids": ["x", "c"], "latency_ms": 20},
    {"relevant_chunk_ids": ["d"],
     "retrieved_chunk_ids": ["x"], "latency_ms": 100},
]
metrics = retrieval_metrics(samples)
print("retrieval =", metrics)
assert metrics["hit_at_1"] == 0.3333
assert metrics["recall_at_5"] == 0.6667
assert metrics["mrr"] == 0.5

audit = question_metrics([{
    "question_type": "single_choice", "stem": "选择正确项",
    "options": [{"key": "A"}, {"key": "B"}],
    "correct_answer": ["A"], "sources": [{"chunk_id": "demo"}],
    "status": "draft",
}])
print("audit =", audit)
assert audit["schema_valid_rate"] == 1.0
```

retrieval_metrics 使用固定 ID 列表，独立验证指标算法。

只有 A/B 的审计候选能返回 schema_valid_rate=1.0，用来说明审计与生产门禁不同。

本代码不创建或启用这道题，不会改变真实题库。

结果解释：Hit@1 为 0.3333，Recall@5 为 0.6667，MRR 为 0.5，P50/P95 为 20/100；审计结果的 1.0 仅表示当前审计规则。

8.9 真实记录复盘：高分结果应怎样阅读?

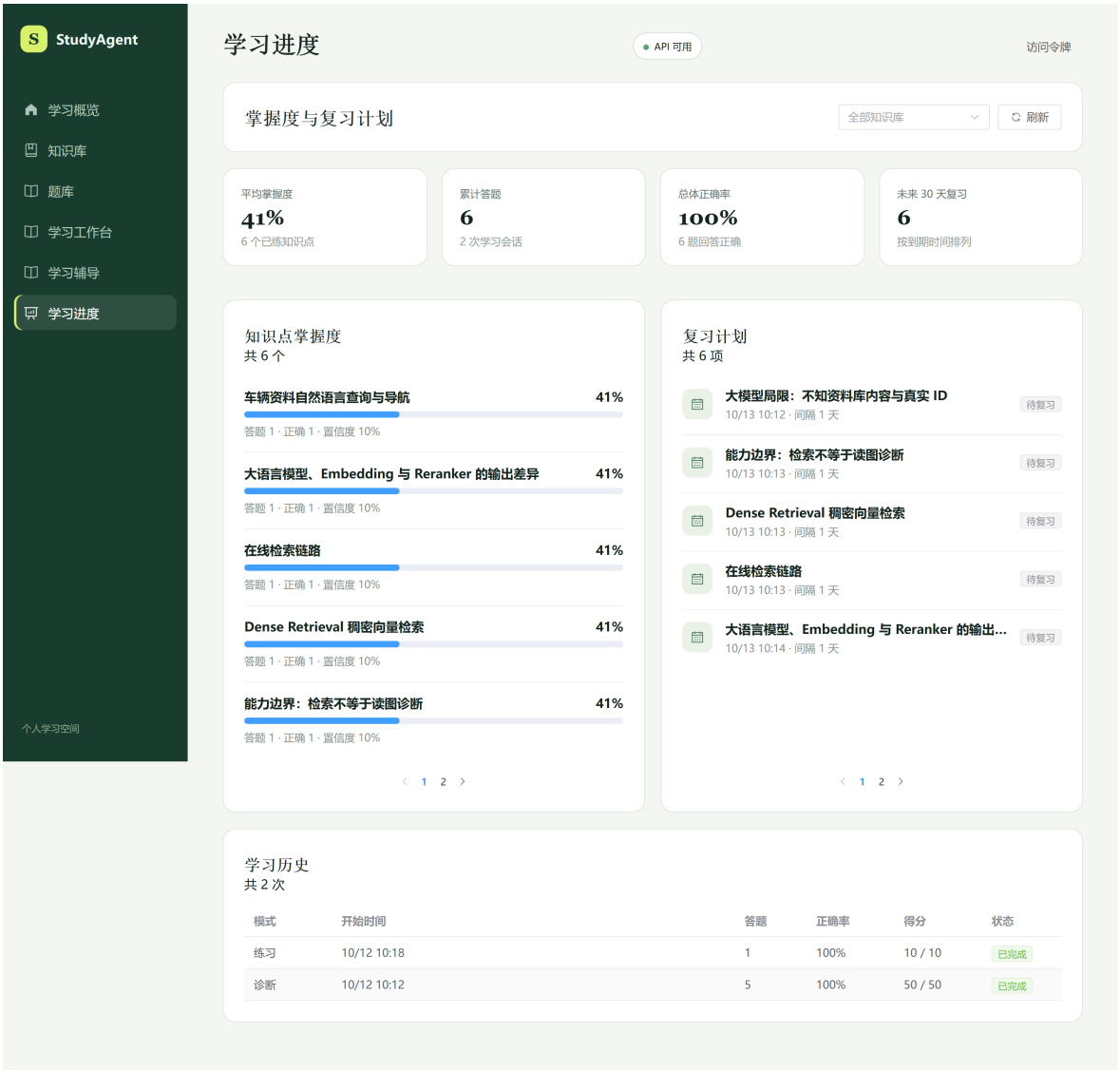


图 8-8 2026-10-12 真实学习进度：名称来自已有知识点，掌握度和复习各显示最多 5 条；界面数字只代表截图时可见业务记录。

当前真实项目页面 / 只读现有业务数据 / 2026-10-12。界面数字只代表截图时的可见记录；没有开始学习或提交出题。

仓库保存了 2026-09-28 的 30 条 TCP 检索历史结果：text-embedding-v4，1024 维，Hit@1/3/5、Recall@5、MRR 均为 1.0，P50 399.901 ms、P95 600.246 ms。但候选语料只有可靠传输、三次握手、四次挥手三个片段，30 条查询主要是同义表述，不能外推为长语料的通用召回率。

同日 59 道题审计记录为结构审计率 1.0、来源关联率 1.0、精确重复率 0.0，题型分别为单选 24、填空 16、判断 19；状态为 active 3、draft 56。人工审核清单仍需要填写。这些指标说明机器可检查的结构与关系，不代表 59 道题都具有充分依据和良好区分度。本文没有再次执行这些历史评测。



图 8-9 从可复现输入走向改进结论，而不是只截图一个漂亮数字。

复盘要优先找失败类型：相关段没召回，先检查切块与检索；召回正确但回答扩展无依据，先检查回答约束；题目结构合格但答案不充分，先检查语义审核。只有扩展跨主题资料、易混淆负例和跨章节问题后，才有依据决定是否增加重排序或混合检索，不能仅按课程组件数量升级架构。

2026-10-09 的新增验收证据

方案 B 的合成资料真实模型验收：目标 10、通过 10、拒绝 0，并完成来源序列化、题库索引、启用、作答、掌握度和复习检查。数据为单份合成样本；不能外推成所有 PDF 的通过率或正确率。本次教材仅引用已有结果，不重新请求模型。

8.10 读懂进度页：真实名称、每页 5 条与累计统计

知识点掌握度和复习计划现在显示 KnowledgePointModel.name，而不是“知识点 + 编号前八位”。/api/mastery 和 /api/reviews/due 在查询时关联真实名称，历史诊断记录无需重做；它们没有请求模型重新生成名称。

列表 / 指标	当前展示方式	数据范围与翻页关系
知识点掌握度	每页 5 条，独立翻页	接口仍返回当前范围的全部掌握度，前端切分显示
未来 30 天复习计划	每页 5 条，按到期时间排序	接口按未来 30 天截止时间查询，前端独立翻页
学习历史	每页 5 条，后端分页	只请求当前页；会话总数来自响应 total
平均掌握度 / 累计答题 / 总体正确率	顶部统计不随分页变化	由全部掌握度记录计算；不是当前 5 行的平均或合计
知识库筛选与刷新	回到各列表第 1 页	切换范围后忽略旧请求的迟到响应，避免残留旧列表

表 8-6 分页限制页面长度；不会改变数据库中的学习记录与掌握度规则

长名称会截断显示，鼠标停留可查看完整名称。名称暂缺时显示“未命名知识点”，不会把数据库 ID 当作人类可读名称。掌握度与复习名称来自知识点表；重新命名知识点后，已有学习记录的下一次查询也会带上当前名称。

源码节选：查询掌握度时一次关联真实名称

定位：apps/api/study_agent/infrastructure/repositories.py，第 917-936 行。

```
917     async def list(self, knowledge_base_id: UUID | None) -> Sequence[MasteryRecord]:
918         # 一次关联取得名称；已有学习记录也立即使用当前知识点名称。
919         stmt = select(MasteryRecordModel, KnowledgePointModel.name).outerjoin(
920             KnowledgePointModel, MasteryRecordModel.knowledge_point_id == KnowledgePointModel.id
921         ).where(
922             MasteryRecordModel.knowledge_base_id.in_(
923                 select(KnowledgeBaseModel.id).where(KnowledgeBaseModel.status != "deleted")
924             )
925         )
926         if knowledge_base_id:
927             stmt = stmt.where(MasteryRecordModel.knowledge_base_id == knowledge_base_id)
928         stmt = stmt.order_by(
929             MasteryRecordModel.mastery_score,
930             MasteryRecordModel.updated_at.desc(),
931             MasteryRecordModel.knowledge_point_id,
932         )
933         return [
934             _mastery_from_model(model, name)
935             for model, name in (await self.session.execute(stmt)).all()
936         ]
```

outerjoin 关联知识点名；保留原掌握度记录及已删除知识库过滤。

_mastery_from_model 把 knowledge_point_name 带入响应，不改变判分和掌握度存储。

全答对为什么仍显示 41%

一次正确的更新是 0.30 + 0.15 × (1 - 0.30) = 0.405；页面四舍五入显示 41%。正确率说明本次记录答对的比例；掌握度是逐次更新的保守规则估计。每点只答一道题时，confidence 只有 10%，不等于已经证明稳定掌握，也不能直接说是“错题薄弱点”。

8.11 练习、参考思路与后续学习

把本章的公式、代码和真实页面连起来：业务计数来自数据库，算法参数来自源码，评测数字来自带输入集与时间的结果文件。三者都需要说明上下文。掌握度规则本身也可以评估，例如是否总把某个知识点反复选中、错误后是否提供合理难度，但当前没有可填写的学习效果提升百分比。

- 练习一：让离线作答顺序改成错误、正确、正确，手算每次掌握度和复习间隔。
- 练习二：把候选题的来源从一条改成两条，算出题目分与总分变化，并说明为什么不等于语义更正确。
- 练习三：构造一个前五条只命中一项、标注有三项的检索样本，比较 Hit@5 与 Recall@5。

• 练习四：查看历史 JSON 的模型、数据集哈希、样本数与延迟，写出一条结论和两条不能外推的判断。

参考思路：错误先使 M=0.24，后续正确按新的存储值更新；复习错误归零后重新走 1 天、6 天。来源数从一变二使 S 从 0.5 变 1，题目分增加 0.1，总分增加 0.045。单次命中可以让 Hit 为 1，Recall 只有 1/3。结论必须限定数据集和候选范围，并把人工审核欠缺写入复盘。

后续扩展顺序

先补足跨主题数据和人工内容审核，再评估参数调整；需要时再引入 RAGAS、重排序或混合检索。算法版本化、真正滑动窗口和独立学习效果评估属于后续任务，不写成当前完成能力。

8.12 综合练习与参考思路

练习 1：模型讲解失败会把已经正确的答案判错吗?

参考思路：不会。成绩由程序确定并先提交，解释失败使用题库解析回退。

练习 2：recent_accuracy 和 confidence 当前分别表示什么?

参考思路：前者是累计正确率，后者是答题次数 / 10 的饱和值；都不是统计置信区间。

练习 3：同一知识点的追问惩罚是否是禁止再次选题?

参考思路：不是。达到已选三题且刚答错时扣分，仍可能在其他题不可用时被选中。

练习 4：MRR 是否只看前五个结果?

参考思路：当前实现寻找完整返回列表的首个相关项；Recall 与 Hit 有各自截断范围。

练习 5：为什么 30 条查询全命中仍不能代表通用 RAG 质量?

参考思路：候选语料仅三个 TCP 片段，查询主题有限，没有覆盖跨主题负例、长资料和跨章节问题。

8.13 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/application/study.py	grade_question / StudyService	核对判分、事务顺序、幂等、掌握度和复习公式
apps/api/study_agent/application/adaptive_selection.py	AdaptiveQuestionSelector.select	核对模式过滤、优先级权重与理由字段
apps/api/study_agent/application/study_explanations.py	LangChainStudyExplanationGenerator / fallback_explanation	区分程序成绩、模型讲解和回退证据
apps/api/study_agent/evaluation/metrics.py	retrieval_metrics / question_metrics	核对指标算法与宽松审计边界
apps/api/tests/test_study_grading.py	test_fill_blank_grading_normalizes_width_case_and_punctuation	核对用户输入归一化示例
apps/api/tests/test_adaptive_learning.py	test_practice_prioritizes_weak_knowledge_point	阅读弱点与到期模式离线案例
evaluation/results/2026-09-28-TCP检索评测-text-embedding-v4-30条.json	metrics / cases	检查历史小样本条件，不外推为生产能力
evaluation/results/2026-09-28-P8题库审计-59道.json	metrics	比较机器审计指标与人工审核范围
apps/api/study_agent/infrastructure/repositories.py	SqlAlchemyStudySessionRepository / SqlAlchemyMasteryRepository / SqlAlchemyReviewTaskRepository	已删除知识库如何从历史学习列表中被过滤
apps/web/src/views/ProgressView.vue	visibleMastery / visibleReviews / loadHistory	核对每页 5 条、分页重置和累计统计
apps/api/tests/test_deleted_library_progress_postgres.py	test_progress_responses_use_current_point_names_for_existing_records	在隔离数据库验证旧记录能取得当前名称
apps/web/tests/e2e/progress.spec.ts	分页与知识库切换测试	检查超过 100 次会话时的总数和翻页行为

下一章：八章主线完成后，先运行离线实验与真实页面观察清单，再选择数据扩展、人工审核或自定义 LangGraph 专项练习。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。